

Privileged, but Biased: How PI-Conditioned Teachers Break Self-Distillation

Sarthak Harne Chinmay Karkar Yash Pandya Ahmed Awadallah Akshay Nambi*
Microsoft Research

Abstract

Self-distillation (SD) has emerged as a compute-efficient alternative to reinforcement learning with verifiable rewards: a self-teacher, conditioned on privileged information (PI) about the answer such as a reference solution, supplies dense per-token supervision to a student that never sees it. Reported gains, however, come almost exclusively from narrow, low-difficulty settings, leaving open a basic question: as a lone objective, with no reward term, does SD teach anything? We reproduce SDPO’s reported gains in its easy setting, then apply the identical setup to difficult tasks and find that it does not. Across question answering, mathematics, coding, and multi-turn agentic tool use, across reasoning modes, model sizes, and forms of PI, and under both the SDPO and OPSD recipes, the per-token loss falls steadily while validation accuracy does not improve and typically degrades. We explain this failure through a single causal chain from the loss to the model it produces. The chain begins with PI bias: having seen one particular reference solution, the teacher’s per-token target is pulled toward that trajectory rather than toward correctness in general, an effect we quantify with a PI Bias Score. Trained to match this target everywhere, the student’s objective becomes nearly blind to whether a rollout is correct, and the loss it assigns falls mostly on low-information tokens like stopwords, punctuation, uncertainty markers, rather than those that determine the answer; within correct rollouts the exploratory tokens incur the highest divergence, so it penalizes the hesitation that reasoning requires. The result is a flatter, less decisive student that is no better at reasoning: as a lone objective, SD optimizes a signal decoupled from task success.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) is a standard recipe for post-training large language models to reason [Shao et al., 2024, Lambert et al., 2024], but its supervision is coarse, sparse, and expensive: one scalar per rollout gives every token the same credit, many prompts yield no correct rollout, and each update needs a verifier and many on-policy samples. This motivates a denser signal supplied *during* the rollout. Distillation [Ko et al., 2024, Agarwal et al., 2024] supplies one but needs a separate, larger teacher; SD removes that requirement by building the teacher from the model being trained, conditioning it on privileged information (PI) the student is not given, such as a reference solution, a hint, or execution feedback [Shenfeld et al., 2026]. Conditioned on the PI the model is a much stronger next-token predictor of a correct trajectory, so matching student to teacher along the student’s rollout turns a single privileged example into a dense per-token signal, with no separate teacher and no reward. SDPO [Hübotter et al., 2026] and OPSD [Zhao et al., 2026] use SD in RL post-training and report matching or beating GRPO at lower sample and compute cost, but on comparatively easy tasks such as short multiple-choice knowledge questions. **On difficult tasks, does SD as a lone objective, with no reward term, optimize task correctness, or a proxy that merely correlates with it easy tasks?**

*Corresponding author: akshayn@microsoft.com

We find that it does not optimize task correctness. We reproduce the reported SDPO behavior in its easy setting, then apply the identical setup to difficult tasks under four varied domains: general QA (MMLU-Pro), mathematics (DAPO-Math), coding (CodeForces), and multi-turn agentic tool use. We observe the opposite outcome: the per-token loss decreases steadily while validation accuracy does not improve and typically degrades, by up to 3.51 points on average in the agentic domain and 7.0 on individual benchmarks, under both recipes (Figure 2, Section 5.2). Since code, teacher, and clipping are unchanged, the difference lies in the objective, and not in the specific recipe.

The cause is two design choices the methods share: **PI-conditioning and per-token density jointly decouple the loss from correctness**. A teacher that has read one reference solution defines at each position a distribution shaped by that solution’s surface form, not by correctness, and the dense divergence targets every token, including the many that do not decide the answer. Our central contribution makes this concrete as a **causal chain**: PI bias $\rightarrow$ a loss blind to correctness $\rightarrow$ effort spent on uninformative tokens $\rightarrow$ exploration penalized $\rightarrow$ a flattened student, every link measured on the same Qwen3-8B runs (Section 6, Figures 3 and 4).

(1) The teacher encodes one solution, not correctness. Our *PI Bias Score*, a teacher-minus-student log-probability ratio needing only forward passes, reaches 0.52 on the in-context solution against ≤ 0.02 on a *different correct* solution to the same problem. This shows that the teacher barely separates from a solution to an unrelated problem. What the privileged information transfers is a specific trajectory instead of correctness.

(2) A target built from one trajectory says nothing about the trajectory correctness. Split by verifier outcome, per-token loss ($\approx 3.5 \times 10^{-4}$) and KL (≈ 0.03) overlap for correct and incorrect rollouts at every step. When they differ, it runs backwards, the teacher–student gap settles at 0.7 on correct rollouts against 0.45 on incorrect ones. This indicates that the learning pressure is highest where the student is already right, so the loss falls without accuracy following.

(3) Density spreads that signal over tokens that do not decide the answer. Stopwords, uncertainty markers, punctuation and whitespace absorb 55.4% of the per-token loss, while the content words, numbers and math symbols that fix the answer absorb comparatively little.

(4) Where the loss is high, it penalizes exploration. Within *correct* rollouts, positions off the reference path carry KL out to ≈ 0.31 against ≈ 0.08 on-path, a factor of four. Having read the solution, the teacher cannot tell a productive detour from a mistake, so the search that reasoning requires is penalized shortening the model responses.

(5) The student flattens rather than sharpens. The teacher–student gap closes from ≈ 3.1 to ≈ 0 , but its ± 1 standard-deviation band does not contract and student entropy rises. The endpoint is a flatter, earlier-committing policy that solves no more problems. This becomes the training signature of Figure 2.

Prior work saw fragments of this from opposite ends: RLSD [Yang et al., 2026a] derives a conditional mutual-information *leakage* bound but stops at theory, while Kim et al. [2026] document degraded reasoning but leave its origin open; we show these are two ends of one mechanism, on difficult tasks where the method is actually used.

Our contributions are: (1) evidence that SD, effective on easy tasks, fails as a lone objective on difficult reasoning, math, coding, and agentic tasks, holding across response length, task difficulty, model scale, reasoning mode, PI form, and both recipes (Sections 5, 7); (2) a causal chain of token- and distribution-level measurements explaining the failure (Section 6); and (3) the *PI Bias Score*, which measures how strongly a PI-conditioned teacher favors the in-context solution over other correct ones from forward passes alone. These reframe SD as an underspecified family of objectives rather than a method that does not work: the density that motivates it is intact; instead the target fails to encode correctness.

2 Background and Related Work

SD sits between two signals it tries to combine: RLVR gives a coarse trajectory-level signal, while distillation gives a dense per-token signal but needs an external teacher. RLVR trains π_θ to maximize a binary reward $R(x, y)$ checking whether response y to prompt x is correct [Lambert et al., 2024], and GRPO [Shao et al., 2024] estimates a group-relative advantage from G rollouts per prompt with a

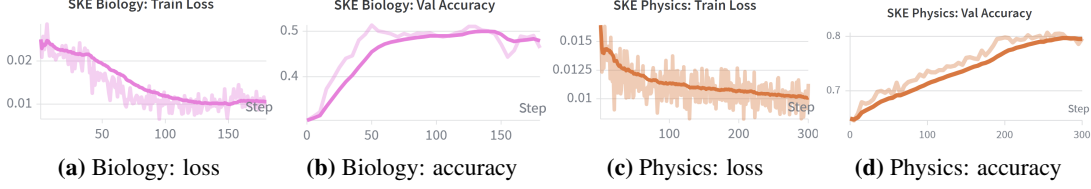


Figure 1: Reproducing SD in its original regime. Qwen3-8B on SciKnowEval with whole-solution PI. Loss decreases and validation *rises* on Biology and Physics, matching the reported SDPO trend.

PPO-style clipped update; every token receives the same advantage, and it vanishes when all rollouts in a group share a reward.

Self-distillation: SDPO and OPSD. SD replaces the reward with PI, side information about the answer available at training but not at test time; we use “PI” as an umbrella term for the feedback f of SDPO [Hübner et al., 2026], the reference solution y^* of OPSD [Zhao et al., 2026] and RLSD [Yang et al., 2026a], and the conditioning context c of Kim et al. [2026]. Training minimizes a stop-gradient per-token divergence between teacher and student along a student rollout (Eq. (1)), so no external model or reward is required. SDPO uses a per-token KL to a teacher conditioned on f , ranging from a prior successful rollout to environment output such as failed unit tests and stabilized by an EMA or trust-region teacher; it matches or exceeds GRPO at several-fold lower compute on SciKnowEval, ToolAlpaca and LiveCodeBench. OPSD uses a forward KL($p_T \| p_S$) with the teacher conditioned on y^* and fixed to the initial policy, observes that a few stylistic tokens carry far higher divergence than answer-determining ones, casts the update as a dense token-level rather than sequence-level policy gradient, and reports gains over GRPO and off-policy distillation on AIME [Mathematical Association of America, 2024, 2025] and HMMT [Dekoninck et al., 2026].

Critiques and augmentations. RLSD [Yang et al., 2026a] proves the objective decomposes as $\mathcal{L}_{\text{OPSD}} = \mathcal{L}^* + I(Y_t; R \mid X, Y_{<t})$, an irreducible conditional mutual information: the teacher fits a single PI-conditioned trace rather than the marginal over valid solutions, so a teacher favoring one phrasing trains the student to adopt it; RLSD’s fix uses the teacher only to reweight a verifier-driven advantage. Kim et al. [2026] report that richer teacher conditioning makes reasoning more concise and confident, speeding in-domain optimization but degrading out-of-distribution and on harder problems by up to $\sim 40\%$ across the Qwen3 and DeepSeek-R1-Distill families. A further line keeps a verifiable reward primary and enters SD as a small-weight auxiliary regularizer [Wang et al., 2026, Lu et al., 2026, Yang et al., 2026b] in multi-turn and agentic settings. Across all of this work SD as a *lone* objective on difficult tasks is never tested; that is the gap we take up, tying the leakage and uncertainty-suppression accounts to one set of measured training runs.

3 The SD Objective and Two Sources of Bias

A policy π_θ generates $y = (y_1, \dots, y_T)$ for a prompt x autoregressively. The student is conditioned on the prompt alone and the self-teacher is the same model additionally conditioned on the PI r : $p_S(\cdot \mid x, y_{<t}) = \pi_\theta(\cdot \mid x, y_{<t})$ and $p_T(\cdot \mid x, y_{<t}) = \pi_{\theta'}(\cdot \mid x, r, y_{<t})$, where θ' is the student’s own parameters or a slow copy that tracks them. Conditioning on r makes p_T a stronger next-token predictor of a correct continuation; the student, which never sees r , is trained to match it along a rollout $\hat{y} \sim p_S$ from the student itself:

$$\mathcal{L}_{\text{SD}}(\theta) = \mathbb{E}_{(x,r)} \mathbb{E}_{\hat{y} \sim \text{sg}(p_S)} \left[\frac{1}{|\hat{y}|} \sum_{t=1}^{|\hat{y}|} D(\text{sg}(p_T(\cdot \mid x, r, \hat{y}_{<t})) \parallel p_S(\cdot \mid x, \hat{y}_{<t})) \right], \quad (1)$$

where sg is stop-gradient, so gradients flow only through the student. SDPO and OPSD instantiate D differently; we study Eq. (1) as a **lone objective**, with no reward term. Two of its properties drive everything that follows.

Property 1: a dense per-token target. Unlike a trajectory-level reward, Eq. (1) imposes a target at every position; both SDPO and OPSD analyze this as a dense token-level policy gradient (SDPO via its per-token KL, their Proposition 2.1, and OPSD by contrast with sequence-level self taught

reasoning). Density is the design’s advantage, but it also places punctuation, stopwords, and style-carrying discourse markers on equal footing with content-bearing tokens.

Property 2: a PI-conditioned target. With a whole reference solution $r = y^*$ the target reflects one particular solution, in both the answer it reaches and how it is phrased and ordered; matching it token by token asks the student to reproduce that behavior everywhere. RLSD [Yang et al., 2026a] formalizes the cost as an irreducible $I(Y_t; R \mid X, Y_{<t})$, measuring how much the target depends on the PI beyond what correctness requires. A dense loss aimed at one solution’s surface form therefore spreads learning signal across all tokens rather than the decisions that make an answer correct. We explore this further link by link in Section 6.

4 Experimental Setup

Domains and data. Prior work evaluates SD on short, knowledge-recall tasks. To test the objective where it is meant to be used, we assemble four harder domains: general QA uses MMLU-Pro [Wang et al., 2024], extending SDPO’s SciKnowEval format to ten options and reasoning over recall; mathematics uses the de-duplicated DAPO-Math-17k [Yu et al., 2025, Open R1 Team, 2025b], whose spread of response lengths separates trace length from difficulty (Section 7); coding uses CodeForces [Open R1 Team, 2025a], demanding an algorithm rather than a short completion; and the agentic setting uses BFCL [Patil et al., 2024], requiring state across turns and composed tool calls. We train on 2,000 examples for general QA, math, and coding, and on the BFCL multiturn split, evaluating every domain **in-domain** and on **held-out transfer benchmarks** from different sources, so a gain cannot come from fitting the training distribution. Transfer benchmarks are SciKnowEval [Feng et al., 2024] and GPQA-D [Rein et al., 2024]; AIME24 [Mathematical Association of America, 2024], AIME25 [Mathematical Association of America, 2025], Olympiad Bench [He et al., 2024]; MBPP+ [Austin et al., 2021], HumanEval+ [Chen et al., 2021], CodeElo [Quan et al., 2025], LCBv6 [Jain et al., 2025]; and BFCLv4 multiturn [Patil et al., 2024]. Data details are in Appendix A, prompts and templates in Appendix D.

Models and privileged information. Our primary model is Qwen3-8B [Yang et al., 2025] in both its think (long chain-of-thought) and instruct variants, so every domain is studied under both reasoning modes; for a size comparison we additionally train Qwen3-32B on general QA. Our default PI is the whole reference solution $r = y^*$: following Hübötter et al. [2026] we sample multiple rollouts per query and take one random correct rollout. Section 6.1 additionally studies short hints (one to two sentences) and skills (≈ 500 -token structured guides of relevant techniques).

Objective, divergence, and training. We train with the lone objective of Eq. (1), no reward or verifier in the loss, instantiating D as the symmetric Jensen–Shannon divergence, $\text{JSD}(p \parallel q) = \frac{1}{2}\text{KL}(p \parallel m) + \frac{1}{2}\text{KL}(q \parallel m)$ with $m = \frac{1}{2}(p + q)$, matching SDPO’s implementation; the OPSD variant is reported in Section 5.2. Two ingredients stabilize training on harder data, both motivated by the loss concentration of Section 6.3: aggressive per-token clipping of the divergence [Zhao et al., 2026], preventing a few high-divergence stylistic tokens from dominating the gradient, and a slow EMA teacher $\theta' \leftarrow (1 - \alpha)\theta' + \alpha\theta$ with $\alpha=0.001$ (vs. 0.01 in SDPO and 0.05 in Kim et al. [2026]), which removed the late-training instabilities we first observed. Hyperparameters: JSD clipping $\tau=0.001$, learning rate $1e-5$, batch size 32, 3 epochs (Appendix B). We report task accuracy (math, QA), fraction of test cases passed (coding), and the BFCL success metric (agentic), each averaged over 4 sampled val trajectories per task at each validation step, and track the per-token SD loss, validation accuracy, student entropy $\mathbb{H}[p_S(\cdot \mid x, \hat{y}_{<t})]$, and average response length, plus teacher–student KL and teacher perplexity for Section 6.5.

Reproducing SD in its original regime. Using whole-solution PI, we train Qwen3-8B with Eq. (1) on SciKnowEval with SDPO’s codebase [Hübötter et al., 2026] and reported hyperparameters, and recover the qualitative SDPO result: on Biology and Physics the loss decreases while validation accuracy rises and responses shorten (Figure 1), confirming that implementation is faithful. We further add on to the SDPO codebase for all our experiments. Our code is available as part of the supplementary material.

5 The Training Signature of Lone SD

Applying the same implementation to the four domains produces a consistent training signature: the loss is optimized, and nothing that determines task success improves. Harder data also changes where the loss lives. Unclipped, the top 50% of tokens consume 98.5% of the loss on MMLU-Pro, mirroring the skew OPSD [Zhao et al., 2026] reports on math: a few high-divergence tokens dominate the gradient, so the average falls while little signal reaches content tokens. This is what the clipping and slow teacher of Section 4 address.

5.1 Patterns observed during training

Figure 2 shows the training dynamics for Qwen3-8B, and the same pattern recurs across settings. **The loss is optimized:** the per-token JS divergence of Eq. (1) decreases steadily, indicating closer matching to the teacher. **Validation accuracy does not follow:** it remains flat or degrades, so optimization is decoupled from task performance. **Student entropy rises:** $\mathbb{H}[p_S(\cdot \mid x, \hat{y}_{<t})]$ increases, contrary to the sharpening expected of an improving model (Section 6.5). **Response length falls:** while desirable for short-form generation, on reasoning tasks this is also consistent with reduced exploration before committing (Section 6.4). Any single trend admits a benign interpretation; together they indicate a systematic failure mode. The pattern is consistent across all four domains and both reasoning modes. The only exception is agentic instruct mode, where the small training set and short responses (300 tokens) yield a modest validation improvement. Because the domains differ substantially in response length, credit structure, and reference-solution format, the shared behavior is most naturally attributed to the objective rather than any particular dataset. Training curves for all domains are provided in Figure 2.

Figure 2 shows the per-domain training dynamics for self-distillation used as the sole training signal on Qwen3-8B. Each row corresponds to one domain, and each column reports one diagnostic over training: the per-token loss, the validation accuracy, the student entropy, and the mean response length. Across all four domains the per-token loss decreases while the validation accuracy does not improve, and the entropy and response length move in the directions described in the main paper.

5.2 End-task results

No improvement where reasoning is required. Table 1 evaluates each trained model against its own base, in-domain and on transfer. Across the reasoning-heavy domains, in both modes, the objective produces no improvement, and degradation is largest where reasoning demand is highest: the think variant loses 3.51 points on average in the agentic domain, including 7 on BFCL Multi-Turn Missing Function V4, and both modes lose 1.6 across general QA. In-domain and transfer move together, consistent with the objective failing to install a capability rather than installing a narrow one.

Positive averages appear only under the instruct variant. Two domain averages come out positive, neither a gain. The instruct variant improves on the agentic BFCL benchmarks, but BFCL trains on a much smaller split (100 tasks) with short instruct rollouts (mean 300 tokens). In coding, the instruct gain on MBPP+ (+12.6) is offset by losses on HumanEval+ (−3.8), CodeElo (−1.7), LCBv6 (−1.5) and CodeForces (−0.9), so performance still declines averaged over benchmarks and modes. Both cases are confined to the instruct model, where shorter responses limit the reach of the causal chain; under the think, flat or declining is the norm.

The result is not specific to SDPO. We repeat the evaluation under the OPSD recipe, a single rollout per step, fixed correct answers, and an initial-policy teacher. Averages change by −2.0 and −0.7 points on general QA and −4.3 and −0.4 on math (think and instruct), with 18 of 20 benchmark-mode cells non-positive and the same signature throughout (full table in Table 2). Since the recipes differ in divergence direction, teacher schedule, rollout count, and clipping, the behavior tracks the shared structure of Eq. (1) rather than one method’s implementation choices.

What the signature rules out. Three explanations for falling loss and flat accuracy are ruled out above. **Optimization failure:** the loss decreases smoothly under the stabilization of Section 4. **Implementation error:** the same code, teacher, and clipping reproduce the reported gains on SciKnowEval (Figure 1); what changes is task difficulty. What remains is the objective itself.

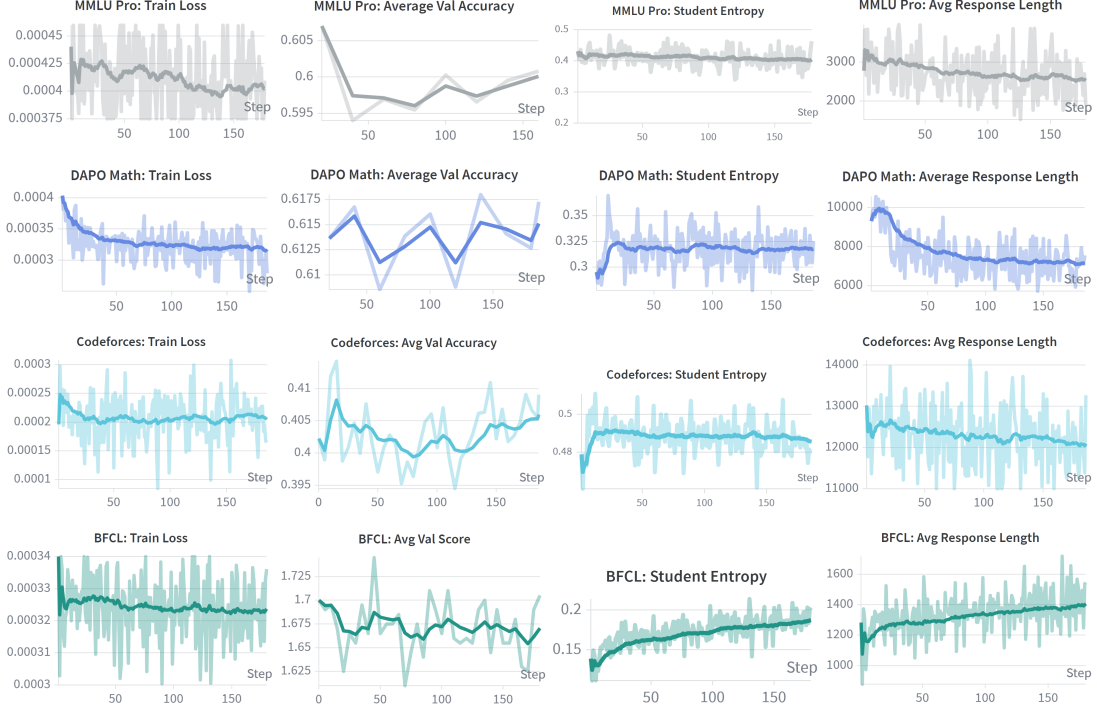


Figure 2: Per-domain training dynamics for lone self-distillation on Qwen3-8B. Rows, from top to bottom, correspond to general QA (MMLU-Pro), mathematics (DAPO-Math), coding (CodeForces), and multi-turn agentic tool use (BFCL). Columns, from left to right, report the per-token loss, the validation accuracy, the student entropy, and the mean response length over training. Across all four domains the loss decreases while the validation accuracy does not improve. For BFCL the second column reports the validation score rather than the accuracy; the score equals twice the accuracy.

6 A Mechanistic Account: From Privileged Information to a Misguided Student

We now measure each link of the predicted chain in causal order, beginning with the one we can intervene on: the bias is a property of the PI, and changing the PI’s specificity moves it as predicted, so the chain does not rest on correlation alone. All measurements use Qwen3-8B with whole-solution PI; a rollout is *correct* if the student solved the problem.

6.1 PI bias: the teacher targets one solution, not correctness

An instrument for the bias. To measure what the teacher moves the student toward, fix a rollout prefix $(x, y_{<t})$ and a target sequence $w = (w_1, \dots, w_K)$. The student and teacher scores are the average log-probability each assigns to w , and the **PI Bias Score** is their difference:

$$\text{PS}_t(w) = s_t^T(w) - s_t^S(w) = \frac{1}{K} \sum_{k=1}^K \log \frac{p_T(w_k \mid x, y^*, y_{<t})}{p_S(w_k \mid x, y_{<t})}. \quad (2)$$

This discounts what the student already assigns to w , reporting only how much the PI moves mass toward it (Appendix E).

A design that separates correctness from imitation. We evaluate Eq. (2) for four targets: the in-context solution y^* , a *different correct* solution y' to the same query, a solution $\tilde{y}$ to an unrelated problem, and an incorrect solution y^- , written PS^* , PS' , $\text{PS}^\sim$, PS^- . The contrast PS^* vs. PS' discriminates: a teacher encoding *correctness* would give $\text{PS}^* \approx \text{PS}'$, one encoding *the trajectory it was shown* would spike on PS^* alone. We observe the second, in every domain (Figure 3): $\text{PS}^* \gg \text{PS}' \approx \text{PS}^\sim > \text{PS}^-$, the teacher barely separating a different correct solution from one to an

Table 1: SD as a lone objective does not improve validation performance across various domains. Positive Δ in green, negative in red; averages over four independent rollouts.

Domain	Benchmark	Think			Instruct		
		Base	SD	Δ	Base	SD	Δ
General QA	MMLU-Pro	70.2 $\pm$ 2.2	69.8 $\pm$ 2.2	−0.4	69.5 $\pm$ 2.1	67.5 $\pm$ 2.3	−2.0
	GPQA-D	59.5 $\pm$ 3.0	57.6 $\pm$ 3.2	−1.9	49.2 $\pm$ 2.7	46.0 $\pm$ 2.8	−3.2
	Bio (SKE)	38.5 $\pm$ 3.4	36.7 $\pm$ 3.6	−1.8	33.5 $\pm$ 3.5	32.3 $\pm$ 3.4	−1.2
	Chem (SKE)	51.9 $\pm$ 2.0	51.9 $\pm$ 2.0	0.0	44.6 $\pm$ 1.8	41.5 $\pm$ 2.0	−3.1
	Mat. (SKE)	68.0 $\pm$ 3.2	64.6 $\pm$ 3.1	−3.4	54.7 $\pm$ 3.1	54.9 $\pm$ 3.4	+0.2
	Phys. (SKE)	77.5 $\pm$ 2.8	75.6 $\pm$ 3.0	−1.9	64.2 $\pm$ 3.2	64.1 $\pm$ 3.3	−0.1
	Average	60.93	59.37	−1.56	52.62	51.05	−1.57
Math	DAPO	79.1 $\pm$ 2.0	80.1 $\pm$ 2.0	+1.0	48.8 $\pm$ 2.2	44.8 $\pm$ 2.4	−4.0
	AIME24	67.5 $\pm$ 7.7	66.0 $\pm$ 7.8	−1.5	24.2 $\pm$ 7.0	23.6 $\pm$ 7.0	−0.6
	AIME25	53.3 $\pm$ 7.4	52.9 $\pm$ 7.4	−0.4	20.0 $\pm$ 6.2	17.2 $\pm$ 6.1	−2.8
	Olympiad	53.6 $\pm$ 1.9	53.4 $\pm$ 1.8	−0.2	46.3 $\pm$ 1.7	46.5 $\pm$ 1.7	+0.2
	Average	63.38	63.10	−0.28	34.82	33.02	−1.80
Coding	CodeForces	24.5 $\pm$ 1.6	25.0 $\pm$ 1.8	+0.5	17.3 $\pm$ 1.3	16.4 $\pm$ 1.3	−0.9
	MBPP+	59.3 $\pm$ 2.3	53.1 $\pm$ 2.1	−6.2	43.2 $\pm$ 2.0	55.8 $\pm$ 2.3	+12.6
	HumanEval+	80.5 $\pm$ 3.1	80.4 $\pm$ 2.9	−0.1	77.4 $\pm$ 2.8	73.6 $\pm$ 3.1	−3.8
	CodeElo	23.8 $\pm$ 1.5	25.1 $\pm$ 1.9	+1.3	13.8 $\pm$ 1.1	12.1 $\pm$ 1.3	−1.7
	LCBv6	50.0 $\pm$ 3.9	53.0 $\pm$ 3.7	+3.0	52.3 $\pm$ 2.8	50.8 $\pm$ 3.2	−1.5
	Average	47.62	47.32	−0.30	40.80	41.74	+0.94
Agentic	Base V3	86.0 $\pm$ 3.2	83.2 $\pm$ 4.6	−2.8	68.0 $\pm$ 4.6	72.5 $\pm$ 4.2	+4.5
	Base V4	51.3 $\pm$ 2.9	48.8 $\pm$ 2.9	−2.5	23.5 $\pm$ 2.8	31.0 $\pm$ 3.2	+7.5
	Miss Func	51.0 $\pm$ 3.0	44.0 $\pm$ 2.9	−7.0	19.5 $\pm$ 2.4	21.5 $\pm$ 2.7	+2.0
	Miss Param	33.0 $\pm$ 2.9	31.36 $\pm$ 2.7	−1.6	17.0 $\pm$ 2.3	19.88 $\pm$ 2.7	+2.9
	Long Ctx	35.5 $\pm$ 2.9	31.88 $\pm$ 2.7	−3.6	16.5 $\pm$ 2.5	23.0 $\pm$ 2.9	+6.5
	Average	51.36	47.85	−3.51	28.90	33.58	+4.68

unrelated problem. **The teacher does not encode correctness; it encodes one solution.** This is the empirical counterpart of the bound of Yang et al. [2026a], turning their irreducible $I(Y_t; R \mid X, Y_{<t})$ into a measurable quantity; the gap $PS^* - PS'$ persists throughout training.

Weaker PI gives a weaker signal, not a better one. If over-specificity were the whole problem, a more general PI should repair it. On DAPO-Math we replace the whole solution with the two weaker forms of Section 4, short **hints** and detailed **skills**. The bias moves as predicted, but not usefully: under both, the score collapses for every target, with PS^* , PS' , PS^- and $PS^\sim$ close together. For hints as PI, the PS^* is 0.0015, PS^- is 0.0192 and $PS^\sim$ is −0.0578, for skills as PI, the PS^* is −0.0021, PS^- is −0.0108 and $PS^\sim$ is 0.0071. Note that for hints and skills, PS^* and PS' do not differ in definition. The teacher no longer singles out the in-context solution, but now moves little mass toward *any* target. Performance follows (Table 3): against a base average of 63.4, hints lose 5.1 points and skills 3.5, while the whole solution stays within half a point of base. The two regimes fail for opposite reasons, which is the sense in which the bias is causal rather than incidental: PI specificity controls the bias and neither end of the dial produces learning, a whole solution tying the target to one trajectory while

Table 2: The same non-improvement pattern holds under the OPSD recipe. Results on General QA and Mathematics using the OPSD training recipe. Positive Δ values are shown in green and negative values in red. All values are averaged over four independent rollouts.

Domain	Benchmark	Think			Instruct		
		Base	SD	Δ	Base	SD	Δ
General QA	MMLU-Pro	70.2 $\pm$ 2.2	70.1 $\pm$ 2.2	−0.1	69.5 $\pm$ 2.1	69.1 $\pm$ 2.3	−0.4
	GPQA-D	59.5 $\pm$ 3.0	57.1 $\pm$ 3.0	−2.4	49.2 $\pm$ 2.7	46.6 $\pm$ 2.8	−2.6
	Bio (SKE)	38.5 $\pm$ 3.4	35.1 $\pm$ 3.5	−3.4	33.5 $\pm$ 3.5	32.6 $\pm$ 3.4	−0.9
	Chem (SKE)	51.9 $\pm$ 2.0	51.2 $\pm$ 2.0	−0.7	44.6 $\pm$ 1.8	43.0 $\pm$ 2.0	−1.6
	Mat. (SKE)	68.0 $\pm$ 3.2	64.2 $\pm$ 3.1	−3.8	54.7 $\pm$ 3.1	56.6 $\pm$ 3.4	+1.9
	Phys. (SKE)	77.5 $\pm$ 2.8	76.1 $\pm$ 3.0	−1.4	64.2 $\pm$ 3.2	63.9 $\pm$ 3.3	−0.3
	Average	60.93	58.97	−1.96	52.62	51.97	−0.65
Math	DAPO	79.1 $\pm$ 2.0	76.0 $\pm$ 1.9	−3.1	48.8 $\pm$ 2.2	46.8 $\pm$ 2.4	−2.0
	AIME24	67.5 $\pm$ 7.7	62.2 $\pm$ 7.3	−5.3	24.2 $\pm$ 7.0	23.7 $\pm$ 7.0	−0.5
	AIME25	53.3 $\pm$ 7.4	48.2 $\pm$ 8.2	−5.1	20.0 $\pm$ 6.2	18.6 $\pm$ 6.1	−1.4
	Olympiad	53.6 $\pm$ 1.9	50.0 $\pm$ 1.8	−3.6	46.3 $\pm$ 1.7	48.4 $\pm$ 1.9	+2.1
	Average	63.38	59.10	−4.28	34.82	34.38	−0.44

hints and skills are too weak to move the student. Examples of hints, skills and how we construct them are detailed in Appendix C.

6.2 The loss does not track correctness

A target defined by one trajectory carries no information about whether the student’s own trajectory reaches the correct answer, so Eq. 1 predicts a loss blind to correctness. An objective that taught problem-solving should assign greater loss to failures. It does not. Mean per-token loss settles near 3.5×10^{-4} and mean KL near 0.03 for both correct and incorrect rollouts, with overlapping four-rollout bands throughout training (Figures 4a, 4b); if anything, incorrect rollouts exhibit slightly *lower* KL. Thus, at no scale does the objective concentrate effort where the student goes wrong. Where the groups do separate, the direction is reversed: the *absolute* log-probability gap $|\log p_T - \log p_S|$ falls

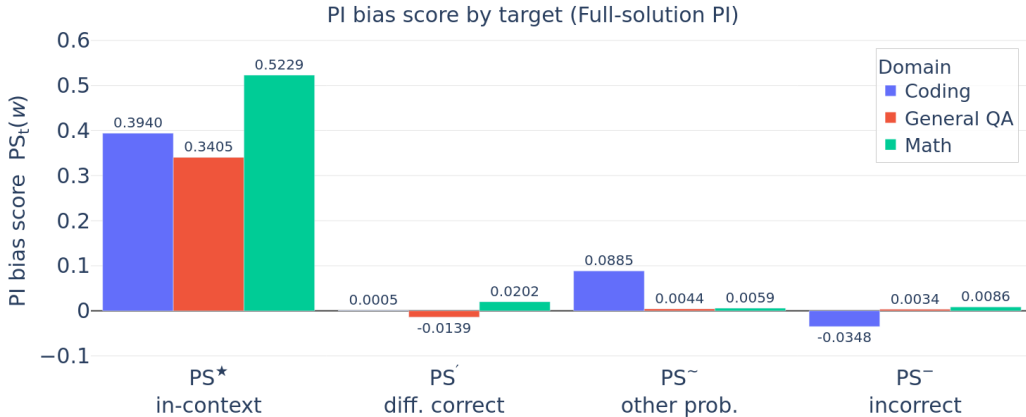


Figure 3: The teacher targets one solution, not correctness. PI bias score (Eq. (2)) per domain, whole-solution PI; taller bar = more mass moved toward that target. Only PS^* , the in-context solution, is tall; a different *correct* solution PS' scores like an unrelated one $PS^\sim$, where a correctness-encoding teacher would give $PS^* \approx PS'$.

Table 3: Neither over-specific nor under-specific PI produces learning. SD on DAPO-Math (Qwen3-8B think) under three forms of privileged information; hints and skills reduce PI bias (Figure 3) but weaken the signal.

Benchmark	Base	Full		Hint		Skill	
		SD	Δ	SD	Δ	SD	Δ
DAPO-Math	79.1 ± 2.0	80.1 ± 2.0	+1.0	75.8 ± 2.1	-3.3	76.3 ± 2.1	-2.8
AIME24	67.5 ± 7.7	66.0 ± 7.8	-1.5	59.0 ± 7.9	-8.5	63.2 ± 8.0	-4.3
AIME25	53.3 ± 7.4	52.9 ± 7.4	-0.4	47.5 ± 8.2	-5.8	49.9 ± 8.1	-3.4
OlympiadBench	53.6 ± 1.9	53.4 ± 1.8	-0.2	50.7 ± 1.8	-2.9	50.3 ± 1.8	-3.3
Average	63.38	63.10	-0.28	58.25	-5.13	59.92	-3.46

from roughly 3 to below 1 within ten steps, then stabilizes near 0.7 on correct rollouts versus 0.45 on incorrect ones (Figure 4c). **The learning signal is strongest on rollouts the student already solves.** This directly explains the decoupling in Section 5: a loss that cannot distinguish success from failure can decrease without improving accuracy.

6.3 The loss lands on uninformative tokens

Per-token density predicts which tokens absorb the loss Section 5 showed to be concentrated. Labelling every token by one of nine mutually exclusive types (special token, whitespace, punctuation, uncertainty marker, stopword, number, math symbol, content word, or other) and attributing each step’s loss accordingly, the tokens absorbing the most are predominantly low-information: in a representative step, stopwords, uncertainty markers (“wait,” “maybe,” “perhaps”), punctuation and

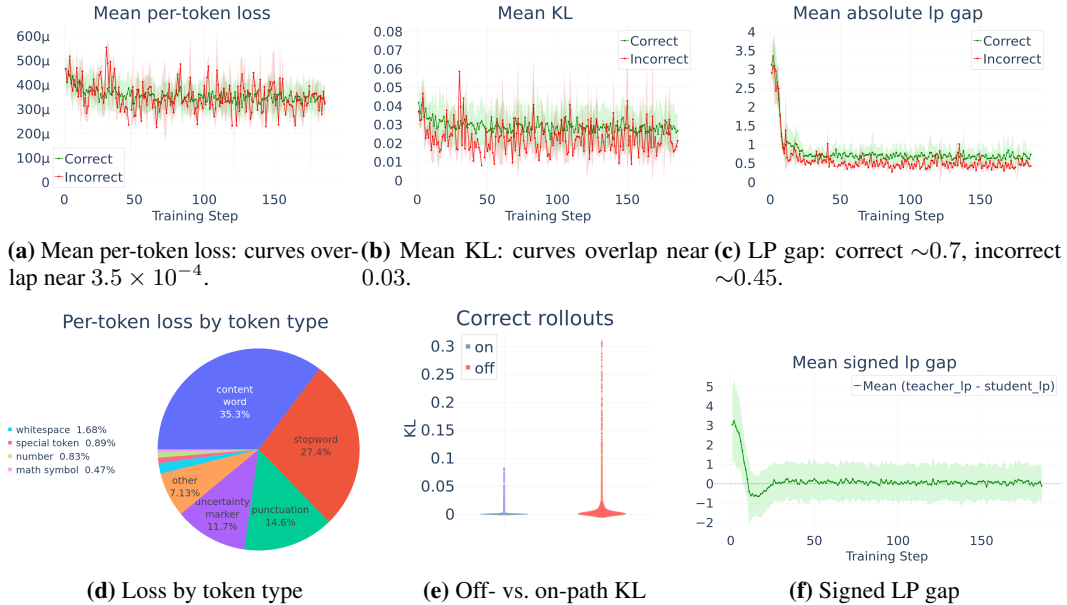


Figure 4: The causal chain. (a–c) **The loss does not track correctness:** for correct (green) vs. incorrect (red) rollouts, loss (a) and KL (b) overlap throughout training, while the absolute LP gap (c) separates them in the *opposite* direction, staying higher on rollouts the student already solves. (d) The loss budget falls mostly on low-information tokens rather than the content words, numbers and math symbols that determine the answer. (e) Even within correct rollouts, off-path exploratory tokens incur much larger divergence than on-path ones. (f) The average signed teacher–student gap shrinks to zero while its variance persists, indicating cancellation rather than convergence.

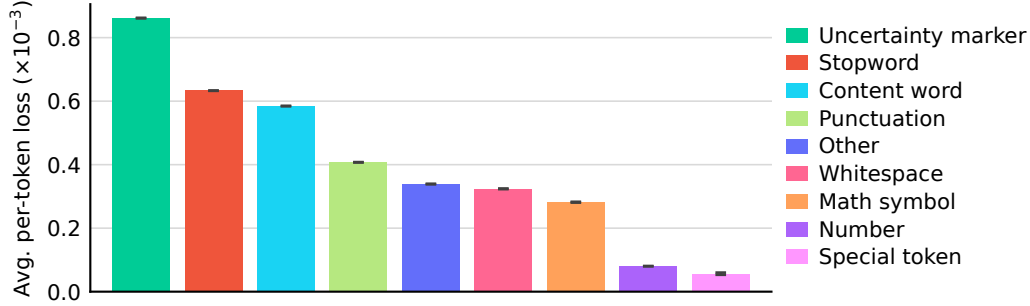


Figure 5: Average per-token distillation loss for each token category. Bars are ordered by decreasing average loss and whiskers denote ± 1 SEM. Because the loss is divided by the number of tokens in each category, a category cannot rank highly merely by being numerous.

whitespace account for **55.38%** of the total per-token loss, while the content words, numbers and math symbols that determine the answer absorb comparatively little (Figure 4d); the ordering is stable across training. Each token is assigned the first category it matches in the order listed above, with the uncertainty-marker set taken from Zhao et al. [2026] and a standard English stopword list. **The objective’s optimization capacity is spent primarily on tokens that do not determine correctness.** This unifies two prior observations: Zhao et al. [2026] that stylistic tokens carry higher divergence than mathematical ones, Kim et al. [2026] that SD shifts epistemic-marker probabilities by an order of magnitude more than the average token. Both are overloaded into one loss budget in which uninformative tokens dominate throughout training, not only at initialization.

Average per-token loss by token type. Figure 5 reports the average per-token distillation loss for each category, computed as the summed per-token loss of a category’s tokens divided by the number of those tokens. Dividing by the token count rather than summing the loss mass means that a category cannot rank highly merely by being numerous. Uncertainty markers carry the highest average per-token loss, followed by stopwords and content words, while numbers and special tokens carry the lowest. Because of this, even when the number of uncertainty markers and punctuations are less, their total loss share is high.

6.4 The objective penalizes exploration

A reasoning model explores several partial paths before committing, so parts of even a correct rollout are legitimately off-path, and a teacher conditioned on y^* cannot recognize such a detour as productive. Ranking rollout positions by the student score $s_t^S(y^*)$, we call the top quartile *on-path* and the bottom quartile *off-path* and compare their full KL distributions, restricting first to correct rollouts so that off-path cannot be conflated with wrong. Within correct rollouts, off-path KL extends to ≈ 0.31 against ≈ 0.08 on-path, a factor of four (Figure 4e). Whereas, within incorrect rollouts the asymmetry widens, ≈ 0.47 against ≈ 0.10 . Because the teacher has read y^* , a detour on the way to a correct answer is indistinguishable from a mistake, and these positions absorb the largest updates. **Within rollouts the student gets right, the objective penalizes exploration most, treating the search that reasoning requires as error.** This supplies the mechanism behind an effect Kim et al. [2026] report only as a phenomenon, and it is why responses shorten: a model penalized for deliberating commits sooner.

6.5 The student flattens rather than sharpens

The accumulated pressure leaves a trace on the student’s distribution. The mean teacher–student log-probability gap falls from ≈ 3.1 at initialization, undershoots to ≈ -0.65 around step 15, and settles at ≈ 0 from roughly step 30 (Figure 4f), even with an EMA rate as small as 0.001. Read alone, a gap of zero says the student has matched its teacher; two observations rule that out. The ± 1 standard-deviation band does not contract with the mean, staying near ± 1 : the student matches the teacher *on average* while disagreeing as much as ever at individual tokens, which indicates cancellation rather than convergence. And over the same period student entropy rises in most domains, with the teacher’s perplexity on student rollouts rising alongside, whereas a student that had genuinely learned

Table 4: Robustness to trajectory length. Response-length tertiles on DAPO-Math (short [1318, 4364], medium [4387, 11939], long [11972, 16384] tokens); Qwen3-8B think. “SD” and $\Delta = \text{SD} - \text{Base}$ follow Table 1, **green** positive and **red** negative; base scores are omitted for space and recoverable as $\text{SD} - \Delta$.

Benchmark	Response Length (Qwen3-8B)					
	Short		Medium		Long	
	SD	Δ	SD	Δ	SD	Δ
DAPO-Math	80.5 $\pm$ 1.9	+1.4	80.1 $\pm$ 1.9	+1.0	79.7 $\pm$ 2.0	+0.6
AIME24	62.9 $\pm$ 7.5	−4.6	65.9 $\pm$ 7.5	−1.6	63.8 $\pm$ 7.4	−3.7
AIME25	52.7 $\pm$ 7.8	−0.6	50.7 $\pm$ 7.5	−2.6	50.6 $\pm$ 7.8	−2.7
Olympiad	53.0 $\pm$ 1.8	−0.6	53.7 $\pm$ 1.8	+0.1	53.1 $\pm$ 1.8	−0.5
Average	62.28	−1.10	62.60	−0.78	61.80	−1.58

the teacher’s distribution would become sharper and easier to predict. **The shrinking gap is not convergence but collapse: the student ends flatter and less decisive than it began**, closing the loop with Section 5: a flatter policy commits earlier (falling length) and solves no more problems (flat validation).

7 Robustness

Response length. A dense per-token loss spreads over more positions as trajectories lengthen, so the failure could be dilution; if it were, the short split would improve. Training separately on DAPO-Math response-length tertiles (short [1318, 4364], medium [4387, 11939], long [11972, 16384] tokens) all three fall below the base average of 63.4, by 1.1, 0.8 and 1.6 points; in-domain DAPO-Math rises in every split (+1.4, +1.0, +0.6) while eight of the nine transfer cells fall and the ninth gains 0.1 in Table 4. The short split, where supervision is least diluted, is no better than the long one, and its token-type loss shares are unchanged.

Task difficulty. The failure could instead reflect problems the student cannot solve at all; if so, the easy split would improve. Splitting MMLU-Pro by the base model’s mean success rate (MSR) at 8 samples into easy ($\text{MSR}@8 \in [0.75, 0.875]$), medium ($[0.375, 0.625]$) and hard ($[0.125, 0.25]$) subsets: all three fall below the base average of 60.9, easy by 3.6 points, medium by 1.4, hard by 1.6 as seen in Table 5. In summary, the model performs no better on the easy split as compared to the hard split.

Model scale. SD’s benefits are reported to grow with model size [Hübotter et al., 2026], so a larger model should improve. Table 5 shows how training Qwen3-32B on MMLU-Pro moves averages by -0.6 points in think and -1.4 in instruct, with the signature unchanged and the PI bias score undiminished. The larger model starts from a higher base, 64.7 against 60.9 for Qwen3-8B on the same benchmarks. With the reasoning modes of Section 5 and the PI-form intervention of Section 6.1, the result is therefore unchanged across trajectory length, task difficulty, model scale, reasoning mode, and PI content.

8 Discussion

What fails is the target, not the density. Not optimization, implementation, data, or scale (Sections 5, 7), but the target in Eq. (1): a PI-conditioned teacher predicts *one* correct continuation and is indifferent between it and the others, so it spreads trajectory information, not correctness, over every token. Density cannot repair a target that cannot distinguish a correct continuation from a well-phrased incorrect one.

Why the objective works on easy tasks. Our account predicts rather than contradicts prior work’s regime of success: where responses are short and the answer is a bounded choice, one reference solution nearly spans the space of correct ones, little exploration exists to suppress, and few tokens are irrelevant to correctness, so imitating one solution (even at high PI bias) is close to learning to be

Table 5: Robustness to task difficulty and model scale. Easy/medium/hard subsets of MMLU-Pro (left) and Qwen3-32B on full MMLU-Pro (right); Qwen3-8B think unless noted. “SD” and Δ = SD – Base follow Table 1, **green** positive and **red** negative; base scores are omitted for space and recoverable as SD – Δ .

Benchmark	Difficulty Split (Qwen3-8B)						Model Scale (Qwen3-32B)			
	Easy		Medium		Hard		Think		Instruct	
	SD	Δ	SD	Δ	SD	Δ	SD	Δ	SD	Δ
MMLU-Pro	69.1 $\pm$ 2.2	−1.1	69.9 $\pm$ 2.2	−0.3	69.5 $\pm$ 2.3	−0.7	78.8 $\pm$ 2.0	+0.1	75.7 $\pm$ 2.1	−1.7
GPQA-D	58.3 $\pm$ 3.0	−1.2	58.7 $\pm$ 3.1	−0.8	57.4 $\pm$ 3.0	−2.1	66.5 $\pm$ 3.0	−1.9	49.9 $\pm$ 3.0	−2.5
Bio (SKE)	32.6 $\pm$ 3.8	−5.9	34.9 $\pm$ 3.8	−3.6	35.6 $\pm$ 3.8	−2.9	35.5 $\pm$ 3.4	−1.0	31.3 $\pm$ 3.3	−3.2
Chem (SKE)	51.8 $\pm$ 2.0	−0.1	51.9 $\pm$ 2.0	0.0	52.2 $\pm$ 2.0	+0.3	57.9 $\pm$ 2.0	−2.0	50.9 $\pm$ 2.0	+0.5
Mat. (SKE)	64.7 $\pm$ 3.1	−3.3	64.6 $\pm$ 3.1	−3.4	64.5 $\pm$ 3.1	−3.5	68.4 $\pm$ 3.1	+0.1	68.7 $\pm$ 3.0	+0.4
Phys. (SKE)	67.5 $\pm$ 3.1	−10.0	77.0 $\pm$ 3.1	−0.5	76.9 $\pm$ 2.9	−0.6	77.3 $\pm$ 2.9	+1.0	68.0 $\pm$ 3.2	−1.8
Average	57.33	−3.60	59.50	−1.43	59.35	−1.58	64.07	−0.61	57.42	−1.38

correct. All three weaken as responses lengthen and answers open up; our results and SDPO/OPSD’s are consistent, differing only in where imitation/correctness cease to coincide.

Implications for practice. A falling SD loss is not evidence of learning; monitor validation accuracy, which separates the two cases. The PI Bias Score screens a candidate PI before training at a cost negligible beside it, forward passes over a handful of prefixes and four targets: a large $PS^* - PS'$ gap means the teacher will transfer a trajectory rather than a skill, a uniformly small score that it will transfer nothing. Nor is PI a free hyperparameter with a benign middle setting: both ends of the specificity dial fail.

Relation to methods that retain a reward, and the open problem. The augmentation literature [Yang et al., 2026a, Wang et al., 2026, Lu et al., 2026] divides by what each term supplies: a verifiable reward carries correctness but nothing about form, the SD term form but almost nothing about correctness. This is what RLSD’s fix accomplishes: reweighting a verifier-driven advantage keeps the density and discards the term carrying the bias. Whether combining them repairs the failure we document, our experiments cannot answer: we do not run that setting. A dense target that *also* encodes correctness remains open; because the bias enters at the first link, the teacher’s commitment to a single trajectory, the constructions with the clearest claim on the source make the target reflect the *set* of correct continuations.

Limitations. Our study is observational, characterizing the objective rather than proposing a remedy. We evaluate one model family (Qwen3) at two scales across four domains and make no cross-architecture claim; results average four rollouts, and some per-domain differences are small relative to run-to-run variation. The PI Bias Score is a log-probability ratio at sampled positions, depends on the choice of $y', \tilde{y}, y^-$, and complements the conditional mutual information analysis of Yang et al. [2026a] without estimating it. Hints and skills are model-generated, so the weak-PI results may partly reflect their construction. We do not claim self-distillation is ineffective in general, only that in our settings a PI-conditioned per-token objective alone gives no learning signal aligned with task correctness.

9 Conclusion

We asked whether a dense PI-conditioned target carries information about correctness when it is the sole training signal. It does not. Across four reasoning-heavy domains, loss decreases while accuracy stays flat or degrades, under both recipes, both reasoning modes, and across trajectory length, task difficulty, and model scale. The PI Bias Score shows why: a teacher conditioned on a single reference solution learns that trajectory rather than correctness. The resulting loss cannot distinguish correct from incorrect rollouts, concentrates on already-correct trajectories, low-information tokens, and off-path exploration, and leaves the student flatter rather than sharper. Weakening the PI fails too, so neither overly specific nor overly weak PI signals correctness. The limitation is not target density but a teacher anchored to one reference trajectory rather than the task objective.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, volume 2024, pages 21246–21263, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/5be69a584901a26c521c2b51e40a4c20-Paper-Conference.pdf.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Jasper Dekoninck, Nikola Jovanović, Tim Gehrunger, Kári Rögnavaldsson, Ivo Petrov, Chenhao Sun, and Martin Vechev. Beyond benchmarks: Matharena as an evaluation platform for mathematics with LLMs. In *3rd AI for Math Workshop: Toward Self-Evolving Scientific Agents*, 2026. URL <https://openreview.net/forum?id=DmPE4byHuN>.
- Kehua Feng, Keyan Ding, Weijie Wang, Xiang Zhuang, Zeyuan Wang, Ming Qin, Yu Zhao, Jianhua Yao, Qiang Zhang, and Huajun Chen. SciKnowEval: Evaluating multi-level scientific knowledge of large language models. *arXiv preprint arXiv:2406.09098*, 2024.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiad-Bench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Jonas Hübötter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.
- Naman Jain, Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, 2025.
- Jeonghye Kim, Xufang Luo, Minbeom Kim, Sangmook Lee, Dohyung Kim, Jiwon Jeon, Dongsheng Li, and Yuqing Yang. Why does self-distillation (sometimes) degrade the reasoning capability of LLMs? *arXiv preprint arXiv:2603.24472*, 2026.
- Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. Distillm: towards streamlined distillation for large language models. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Øyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. Introduces HumanEval+ and MBPP+.
- Zhengxi Lu, Zhiyuan Yao, Zhuowen Han, Zi-Han Wang, Jinyang Wu, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Self-distilled agentic reinforcement learning. *arXiv preprint arXiv:2605.15155*, 2026.

- Mathematical Association of America. AIME 2024: American invitational mathematics examination. https://huggingface.co/datasets/Maxwell-Jia/AIME_2024, 2024.
- Mathematical Association of America. AIME 2025: American invitational mathematics examination. <https://huggingface.co/datasets/opencompass/AIME2025>, 2025.
- Open R1 Team. CodeForces dataset. <https://huggingface.co/datasets/open-r1/codeforces>, 2025a.
- Open R1 Team. DAPO-Math-17k-Processed. <https://huggingface.co/datasets/open-r1/DAPO-Math-17k-Processed>, 2025b. Filtered and de-duplicated English subset of DAPO-Math-17k.
- Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfc1): From tool use to agentic evaluation of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Shanghaoran Quan, Jiayi Yang, Bowen Yu, Bo Zheng, Dayiheng Liu, An Yang, Xuancheng Ren, Bofei Gao, Yibo Miao, Yunlong Feng, Zekun Wang, Jian Yang, Zeyu Cui, Yang Fan, Yichang Zhang, Binyuan Hui, and Junyang Lin. CodeElo: Benchmarking competition-level code generation of LLMs with human-comparable elo ratings. *arXiv preprint arXiv:2501.01257*, 2025.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *Conference on Language Modeling (COLM)*, 2024.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Idan Shenfeld, Mehul Damani, Jonas Hübner, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.
- Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. Skill-SD: Skill-conditioned self-distillation for multi-turn LLM agents. *arXiv preprint arXiv:2604.10674*, 2026.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.
- Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. Self-distilled RLVR. *arXiv preprint arXiv:2604.03128*, 2026a.
- Shuo Yang, Jinyang Wu, Zhengxi Lu, Yuhao Shen, Fan Zhang, Lang Feng, Shuai Zhang, Haoran Luo, Zheng Lian, Zhengqi Wen, and Jianhua Tao. Opid: On-policy skill distillation for agentic reinforcement learning, 2026b.

Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

Contents

A	Datasets, Splits, and PI Construction	17
A.1	Sources and splits	17
A.2	PI construction	17
B	Hyperparameters (Implementation Details)	18
C	Qualitative Examples (Full Solution vs Hints vs Skills)	18
D	Prompts and Templates	20
D.1	Per-domain problem prompts	20
D.2	Self-distillation templates	21
D.3	Privileged-information generation prompts	22
D.4	Skill derivation prompt	23
E	PI Bias Implementation Details	24

This supplement provides implementation details, additional results, and qualitative material for the main paper.

Table 6: Domains, training data, and evaluation benchmarks used in the study.

Domain	Train	In-domain	Transfer
General QA	MMLU-Pro (2k)	MMLU-Pro	GPQA-D, SciKnowEval
Mathematics	DAPO-Math (2k)	DAPO-Math	AIME24, AIME25, OlympiadBench
Coding	CodeForces (2k)	CodeForces	MBPP+, HumanEval+, CodeElo, LCBv6
Multi-turn agentic	BFCLv3 Multi-turn base (100)	BFCLv3 Multi-turn base	BFCLv4 Multi-turn base, Miss. func, Miss. param, Long context

A Datasets, Splits, and PI Construction

The study spans four domains: general question answering, mathematics, coding, and multi-turn agentic tool use. For each domain, one dataset provides training and in-domain evaluation, and a set of held-out datasets measures transfer. Table 6 lists them. This section describes their sources, the train, validation, and test splits, and the construction of the three forms of privileged information (PI).

A.1 Sources and splits

General question answering uses MMLU-Pro [Wang et al., 2024] for training and in-domain evaluation. Transfer is measured on GPQA-Diamond [Rein et al., 2024] and on four SciKnowEval [Feng et al., 2024] subjects (biology, chemistry, materials, and physics). The SciKnowEval subjects are divided into training and test portions by a ninety-ten split.

Mathematics uses the processed release of DAPO-Math [Yu et al., 2025, Open R1 Team, 2025b] for training and in-domain evaluation, and measures transfer on AIME 2024 [Mathematical Association of America, 2024], AIME 2025 [Mathematical Association of America, 2025], and OlympiadBench [He et al., 2024].

Coding uses CodeForces [Open R1 Team, 2025a] problems, restricted to those with a problem description, for training and in-domain evaluation, and measures transfer on MBPP+ and HumanEval+ [Austin et al., 2021, Chen et al., 2021, Liu et al., 2023], CodeElo [Quan et al., 2025], and LiveCodeBench v6 [Jain et al., 2025].

The multi-turn agentic domain uses the Berkeley Function-Calling Leaderboard [Patil et al., 2024]. Training and in-domain evaluation use the version-three multi-turn base v3 category, which comprises 100 tasks. Transfer is measured on the version-four multi-turn v4 categories: base, missing function, missing parameter, and long context.

For the in-domain training datasets, a stratified validation subset of 300 examples is drawn, stratified over pass-rate bins. For the transfer datasets, the validation set is the full dataset. At training time, each in-domain training set is subsampled to a random subset of 2,000 examples drawn without replacement. The multi-turn agentic domain is the exception and uses its full training set.

A.2 PI construction

Three forms of privileged information are used. Any one of them can be supplied to training in place of the others.

Whole solution. For each question, one correct solution is selected from the model’s own successful attempts, with its reasoning trace removed. The selected solution is presented to the teacher through the templates of Appendix D.

Hint. A short insight of one to two sentences is generated from the question and a correct solution by a separate model, and is constrained not to reveal the final answer. The generation prompt is given in Appendix D.

Skill. Problems are grouped into clusters, and one structured skill card of roughly 500 words is derived per cluster and shared across its member problems. Each card names a general technique

Problem Statement

For how many values of k is 12^k the least common multiple of the positive integers 6^6 , 8^8 , and k ?

Please reason step by step, and put your final answer within `\boxed{\}`.

Full Solution (y*)

To determine how many values of k make 12^k the least common multiple (LCM) of the positive integers 6^6 , 8^8 , and k , we begin by analyzing the prime factorizations of the given numbers.

Step 1: Prime Factorization

- $6^6 = (2 \cdot 3)^6 = 2^6 \cdot 3^6$
- $8^8 = (2^3)^8 = 2^{24}$
- $12^k = (2^2 \cdot 3)^k = 2^{2k} \cdot 3^k$

Let's denote $k = 2^a \cdot 3^b$, since any number k that is a factor of 12^k must be composed only of the primes 2 and 3 . Adding any other prime would cause the LCM to include that new prime, which would exceed 12^k , violating the condition.

Skill

Skill: Factor-Constraint Arithmetic Modeling

General approach: Recast the wording into an arithmetic model where each variable, divisor, ingredient count, or function value is controlled by prime factorization, congruence classes, and divisibility closure. Instead of trying random examples, identify what the condition forces on every factor or residue: divisors must inherit the stated property, products split into prime-power choices, totals with parity or multiple restrictions become constrained integer solutions, and multiplicative functions are determined by their values on primes. Optimization then happens on the compressed structure: distribute prime powers among variables, compare divisor-count effects, assign smaller images to more influential prime powers, or compute the unavoidable common divisor by taking the minimum prime-adic valuation across all allowed cases. Throughout, use extremal reasoning only after the feasible arithmetic

Hint

Use prime factorization and the fact that an LCM takes the maximum exponent of each prime across the numbers; $\backslash(k\backslash)$ must supply any missing required prime exponent while not exceeding the target exponents.

Figure 6: A representative mathematics training question (top) shown with the three forms of privileged information derived for it. The whole solution presents a complete correct answer, the skill presents a structured technique card for the question’s cluster, and the hint presents a short insight. In training, each form is supplied on its own to the privileged-information-conditioned teacher, following the templates of Appendix D.

together with its tools, common pitfalls, and the cues that indicate when it applies, and is constrained not to reveal any specific answer. The derivation prompt is given in Appendix D.

B Hyperparameters (Implementation Details)

We report the training configuration used for the self-distillation runs, grouped following the format used in the appendix of the original SDPO paper [Hübotter et al., 2026]. The configuration for the primary Qwen3-8B runs is shared across the four domains; we note where the coding and multi-turn agentic domains and the Qwen3-32B size study differ. The base model is Qwen3-8B [Yang et al., 2025], and every domain is studied in both its thinking (long chain-of-thought) and instruct variants; the configuration below corresponds to the thinking variant. All experiments are run on a single node of eight NVIDIA B200 GPUs.

Two coefficients with similar names are distinguished here to avoid confusion. The divergence used by the loss is controlled by a mixing coefficient for which 0 denotes forward KL, 1 denotes reverse KL, and 0.5 denotes the symmetric Jensen-Shannon divergence; every primary run uses 0.5. The rate at which the teacher tracks the student through an exponential moving average is a separate quantity, set to 0.001; this is the teacher EMA rate referred to in the main paper.

The following domain-specific deviations from Table 7 apply. The coding and multi-turn agentic runs use a learning rate of 1×10^{-6} . The Qwen3-32B size study uses a maximum response length of 10240, a maximum model context of 12288, and tensor-parallel size 4.

C Qualitative Examples (Full Solution vs Hints vs Skills)

This section illustrates the three forms of privileged information on a shared training question. Figure 6 shows a representative mathematics problem together with the whole solution, the derived skill, and the derived hint, each of which can serve as the privileged information supplied to the teacher in a separate training configuration.

Whole-solution form. A single correct solution to the question, with its reasoning trace removed, is presented to the teacher under a header that labels it as the correct solution, followed by the instruction

Table 7: Training configuration for the primary Qwen3-8B self-distillation runs, grouped following the format used by the original SDPO paper [Hübotter et al., 2026]. The configuration is shared across the four domains; the deviations noted below the table apply to the coding, multi-turn agentic, and Qwen3-32B size-study runs.

Parameter	Value
<i>General</i>	
Model	Qwen3-8B
Thinking	enabled
<i>Data</i>	
Max. prompt length	2048
Max. response length	16384
<i>Batching</i>	
Question batch size	32
Mini batch size	32
Number of rollouts	8
<i>Rollout</i>	
Inference engine	vLLM
Temperature	1.0
<i>Validation</i>	
Number of rollouts	4
Temperature	0.6
Top- p	0.95
<i>SDPO loss</i>	
Top- K distillation	100
Distillation divergence	Jensen-Shannon ($\alpha = 0.5$)
Clip advantages	none
Teacher-EMA update rate	0.001
Rollout importance sampling clip	2
<i>Training</i>	
Optimizer	AdamW
Learning rate	1×10^{-5} (constant)
Warmup steps	10
Weight decay	0.01
Gradient clip norm	1.0
<i>Additional settings</i>	
Per-token loss clip τ	0.001
Max. reprompt length	16384
Total epochs	3
Training questions per epoch	2000
Validation frequency	every 5 steps

to solve the original question with the student’s own approach, following the mathematics reprompt template of Appendix D.

Hint form. A one-to-two-sentence insight is presented to the teacher through the hint template of Appendix D. The insight names the governing idea, for example the relevant theorem or identity, without stating the final answer.

Skill form. A structured skill card for the question’s cluster is presented to the teacher as five fields. The layout is

```

Skill: <name>
General approach: <prose description of the technique>
Key tools & formulas: <theorems, identities, algorithms>
Pitfalls & checks: <common mistakes, edge cases, checks>
When it applies: <signals indicating this skill>

```

Listing 1: Rendered skill-card fields.

The card describes a general technique for the cluster and does not reveal the answer to any single problem.

D Prompts and Templates

This section reproduces the prompt and template strings used in training and in the construction of privileged information. Long lines are wrapped for display. Placeholders in braces, for example {problem} and {hint}, are filled at run time. Each block is labelled with a numbered listing caption.

D.1 Per-domain problem prompts

The student is shown a domain-specific prompt that wraps each problem.

```
{problem}  
  
Please reason step by step, and put your final answer within \boxed{}
```

Listing 2: General reasoning and mathematics problem prompt.

```
You are a coding expert. You will be given a coding problem, and you need to  
    write a correct Python program that matches the specification and passes all  
    tests. The time limit is 1 second. You may start by outlining your thought  
    process. In the end, please provide the complete code in a code block enclosed  
    with ``` ``.  
  
{problem}
```

Listing 3: Coding problem prompt.

```
The following are multiple choice questions (with answers) about {subject}. Think  
    step by step and then finish your answer with "the answer is (X)" where X is  
    the correct letter choice.  
Question:  
{question}  
Options:  
A. {option_A}  
B. {option_B}  
...  
Answer: Let's think step by step.
```

Listing 4: MMLU-Pro multiple-choice prompt.

```
Return your final response within \boxed{} and only include the letter choice (A,  
    B, C, or D) as your final response.  
Problem: {problem}  
Options: {options}  
Answer:
```

Listing 5: GPQA prompt.

```

Given a question and four options, please select the right answer. Respond in the
following format:
<reasoning>
...
</reasoning>
<answer>
...
</answer>

For the answer, only output the letter corresponding to the correct option (A, B,
C, or D), and nothing else. Do not restate the answer text. For example, if
the answer is "A", just output:
<answer>
A
</answer>

```

Listing 6: SciKnowEval system prompt.

D.2 Self-distillation templates

During self-distillation the teacher prompt is assembled from a reprompt template that combines the problem, a solution section carrying the privileged information, and an optional feedback section. The privileged information enters through the solution section (a whole solution) or through the hint template (a hint or skill). The default templates are shown first, followed by the per-domain overrides used for mathematics and the multi-turn agentic setting; the overrides label the injected privileged information as a correct solution and close the reprompt with an instruction to use the student’s own approach.

```

{prompt}{solution}{feedback}

Correctly solve the original question.

```

Listing 7: Default reprompt template.

```

Correct solution:

{successful_previous_attempt}

```

Listing 8: Default solution section.

```

The following is feedback from your unsuccessful earlier attempt:

{feedback_raw}

```

Listing 9: Default feedback section.

```

Here is a hint for solving this problem:

{hint}

```

Listing 10: Default hint template.

```

hint_template:
  Here is the correct solution for the above question:

  {hint}
reprompt_template:
  {prompt}{solution}{feedback}

  Now, using your own approach, try to correctly solve the original question.

```

Listing 11: Mathematics reprompt and hint templates.

```

reprompt_template:
  {prompt}{solution}{feedback}

  After understanding this, please try to solve this problem using your own
  approach below.
solution_template:
  Here is the correct solution for the problem:
  ```
 {successful_previous_attempt}
  ```

```

Listing 12: Multi-turn agentic reprompt and solution templates.

D.3 Privileged-information generation prompts

Hints and compressed solutions used as privileged information are generated by a separate model.

```

You are given a math problem and its correct solution. Your job is to extract the
core mathematical trick, principle, or insight that unlocks the solution.

Rules:
- 1-3 sentences maximum.
- Do NOT reveal the final answer or any numerical result.
- Do NOT give step-by-step instructions.
- Do NOT start with "The key insight is" or similar preambles.
- Only state the idea, theorem, formula, or trick itself.

Respond in the following format:
<hint>
[your hint here]
</hint>

Problem: ```{problem}```

Solution:
```
{solution}
```

```

Listing 13: Hint generation prompt.

You are given a math problem and its correct (but verbose) solution. Rewrite the solution as a short, dense derivation.

Rules:

- Include ALL key steps and the final answer.
- Remove all verbose explanations, motivations, headers, and formatting.
- Use mathematical notation directly instead of prose where possible.
- Target length: 3-6 sentences or equivalent equations.
- The compressed solution must be self-contained and verifiable.
- Do NOT omit the final boxed answer.

Respond in the following format:

<compressed>

[your compressed solution here]

</compressed>

Problem: ```{problem}```

Solution:

```

{solution}

```

Listing 14: Compressed-solution generation prompt.

D.4 Skill derivation prompt

The skill form of privileged information is derived once per problem cluster.

You are analyzing a cluster of related {domain} to extract ONE reusable problem-solving *skill* that applies across the whole cluster.

A skill is the Goldilocks middle ground between a one-line hint and a full solution: rich enough to materially guide a solver on ANY problem in this class, yet general enough that it never solves a specific instance. It will be shown to a model as privileged context alongside a long (multi-thousand-token) solution attempt, so it must carry enough substance to actually shape the reasoning.

Hard rules:

- Describe a GENERAL, reusable technique/strategy that plausibly applies to MANY problems in this cluster, not just one.
- Do NOT reveal or compute any specific numeric answer, final result, or option.
- Do NOT walk through the solution of any single problem instance.
- Do NOT give a numbered step-by-step procedure or a fixed solution template.
- Be concrete and substantive: name the actual reasoning moves, representations, formulas, theorems, and checks a strong solver would use.

Produce a structured skill card with these fields:

- name: a short 3-7 word handle for the technique.
- approach: the general line of attack, in flowing prose.
- tools: the key formulas, theorems, identities, representations, or algorithms this technique relies on.
- pitfalls: common mistakes, tricky edge cases, and sanity-checks.
- cues: the textual or structural signals in a problem that indicate this skill applies.

Aim for a substantial, information-dense card (roughly 250-400 words total across the fields).

Listing 15: Skill derivation prompt.

E PI Bias Implementation Details

This section documents how the PI Bias Score and the on-path score used in the main paper are computed. For a student rollout, let the student prefix up to position t be its first t response tokens. For a target token sequence w , define the on-path score

$$s_t(w) = \frac{1}{|w|} \sum_{k=0}^{|w|-1} \log P_{\text{model}}(w_k \mid c, \text{prefix}_{<t}, w_{<k}), \quad (3)$$

the mean log-probability the model assigns to the tokens of w when w is appended after the student prefix. The context c is either the teacher prompt, which contains the privileged information, giving $s_t^T(w)$, or the student prompt, which does not, giving $s_t^S(w)$. The PI Bias Score is the difference

$$\text{PS}_t(w) = s_t^T(w) - s_t^S(w), \quad (4)$$

so that a positive value indicates that the privileged information present in the teacher prompt raises the model’s tendency to continue toward w .

Targets. Four targets w are scored at each evaluated position:

- y^* , the in-context correct solution, that is the privileged information itself, taken from the teacher prompt. For hint-style privileged information that does not embed a full solution, a correct solution to the same question is used with its reasoning trace removed.
- y' , a different correct solution to the same question, which requires at least two distinct correct solutions.
- y^- , an incorrect solution to the same question, taken from an incorrect attempt.
- $\tilde{y}$, an unrelated correct solution drawn from the other questions in the same training step.

Positions and lookahead. Each target is scored in full: the lookahead length is set to $K = 2000$ tokens, which exceeds the length of every target, so no target is truncated. Positions are evaluated at every token, over assistant tokens (and not user tokens) only. Prompts longer than 8000 tokens are skipped. When the student is inside a reasoning block at position t , a closing marker is inserted before the target so that the appended continuation is well formed.

Models. The teacher score s^T is computed with the teacher checkpoint and the student score s^S with the student checkpoint at the corresponding training step. Before the first checkpoint, the base model is used for both roles. The two passes are computed separately and merged by position.

Scoring. The score is obtained from a single forward pass over the assembled prompt and target. At each target position the language-model head produces a distribution over the vocabulary, from which the log-probability of the target token is read, and the on-path score is the mean of these log-probabilities over the target.